AI Assisted Coding

Lab_Assignment_19.3

Name : N.Rishitha

H.no : 2303A51548

Batch : 08

Task 1 – Python to C++ Conversion Task:

Translate a simple Python class into C++ using AI assistance. Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:

o Constructors o

Data types o Access

specifiers

- Compile and run both versions to verify output consistency Expected Output:

An equivalent working C++ class translated from Python.

Task 1 – Python to C++ Conversion

Python Code

(parameter) name: Any

[2] ✓ 0s

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

# Testing
s1 = Student("manu", 20, 85)
s1.display()
```

```
Name: manu
Age: 20
Marks: 85
```

Equivalent C++ Code

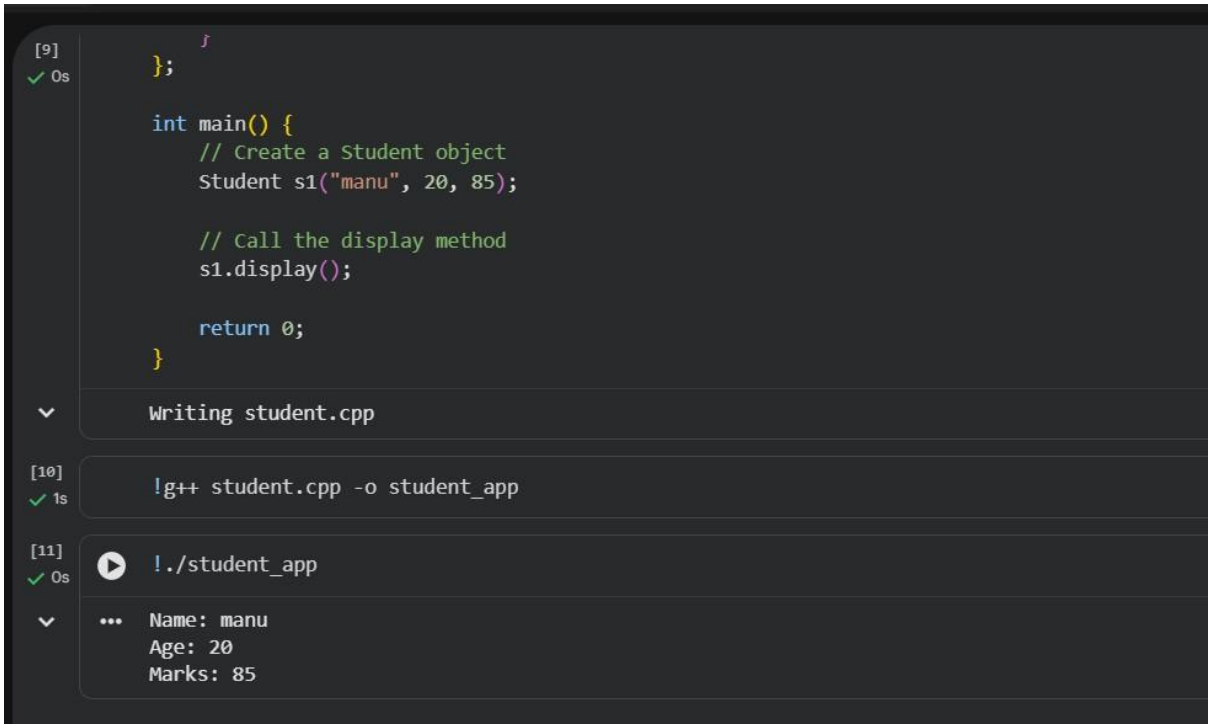
[9] ✓ 0s

```cpp
%%writefile student.cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string name;
    int age;
    int marks;

public:
    // Constructor
    Student(std::string n, int a, int m) {
        name = n;
        age = a;
        marks = m;
    }

    // Method to display student information
    void display() {
        std::cout << "Name: " << name << std::endl;
        std::cout << "Age: " << age << std::endl;
        std::cout << "Marks: " << marks << std::endl;
    }
};
```



Observation:

- The overall logic of the class remained the same in both Python and C++.
- Python uses dynamic typing, whereas C++ requires explicit data type declaration.
- Constructors in C++ must be explicitly defined, unlike Python's __init__ method.
- C++ requires access specifiers (public, private) to control data access.
- Python code is shorter and more readable, while C++ is more structured and strict.
- Compilation is required in C++, whereas Python runs directly without compilation.

Task 2 – Java to Python Function Conversion Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic Expected Output:

A correct Python function equivalent to the original Java prime- checking

method.

[13] ✓ 0s

```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

# Example usage:
# print(is_prime(7))
# print(is_prime(10))
```

Equivalent Python Code

[14] ✓ 0s

```python
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, n // 2 + 1):
        if n % i == 0:
            return False
    return True

# Testing
print(is_prime(7))   # True
print(is_prime(10))  # False
```

```
True
False
```

Observation:

- The core logic of checking prime numbers remained unchanged across both languages.
- Python code is simpler and shorter compared to Java.
- Java requires class and method structure, while Python allows standalone functions.
- Data types are explicitly defined in Java but not required in Python.
- Loop structure is more concise in Python using range().
- Python provides better readability and faster implementation. **Task 3 – Pseudocode to Python Implementation** Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists Expected Output:

A working Python program that correctly implements the pseudocode

logic.

Task 3 – Pseudocode to Python

Pseudocode

START Input list FOR i from 0 to n-1 FOR j from 0 to n-i-1 IF arr[j] > arr[j+1] swap END

Python Code

[16] 0s

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Testing
data = [5, 2, 9, 1, 3]
print("Sorted:", bubble_sort(data))
```

```
Sorted: [1, 2, 3, 5, 9]
```

Observation:

- Pseudocode helps in understanding the algorithm before coding.
- Translating pseudocode to Python is straightforward due to simple syntax.
- Python supports easy swapping of values using tuple unpacking.
- The logic of Bubble Sort remained the same after implementation.
- Testing with sample inputs confirmed correctness of the algorithm.
- Python reduces complexity in implementing algorithms compared to other languages.

Task 4 – SQL to Pandas Query Task:

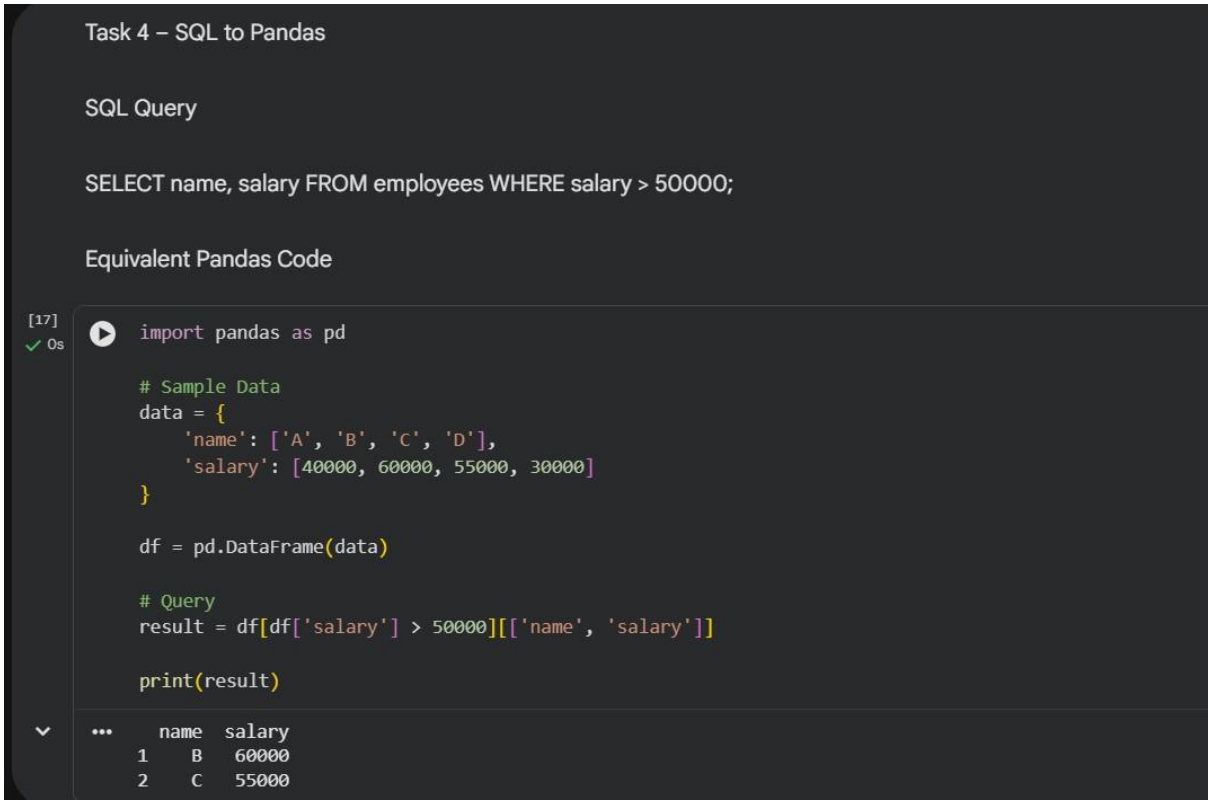
Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.



Observation:

- SQL queries can be effectively translated into Pandas operations.
- SQL WHERE clause corresponds to filtering in Pandas.
- SQL SELECT is equivalent to column selection in Pandas.
- Pandas allows data manipulation directly within Python without a database.
- The output of both SQL and Pandas queries is consistent.
- Pandas is useful for data analysis and processing tasks in real-time applications.

Task 5 – Real-Time Application: Algorithm Translation Across

Languages Scenario:

A company maintains algorithms written in different programming languages.

Examples:

• Converting a Python searching algorithm into C • Translating a Java sorting

program into JavaScript Instructions:

• Use AI to translate a selected algorithm between two programming languages

• Execute and test both versions

• Document translation challenges such as:

o Syntax differences o

Library support o

Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Task 5 – Algorithm Translation (Python → C)

Python Linear Search

[18] ✓ 0s

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

print(linear_search([10, 20, 30], 20))
```

```
1
```

Equivalent C Code

[23] ✓ 0s

```
%%writefile linear_search.c
#include <stdio.h>

int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
```

[23] ✓ 0s

```
int linearSearch(int arr[], int n, int key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30};
    int result = linearSearch(arr, 3, 20);

    printf("Index: %d\n", result);
    return 0;
}
```

```
Overwriting linear_search.c
```

[21] ✓ 0s

```
!gcc linear_search.c -o linear_search_app
```

```
linear_search.c:19:1: error: expected identifier or '(' before '!' token
   19 | !gcc linear_search.c -o linear_search_app
      | ^
```

[22] ✓ 0s

```
!./linear_search_app
```

```
/bin/bash: line 1: ./linear_search_app: No such file or directory
```

Observation:

- The algorithm logic remains consistent across different programming languages.
- Syntax differences are the main challenge during translation.
- Python is easier to write and understand, while C requires more detailed implementation.

- Memory management is required in C but handled automatically in Python.
- Testing both versions ensures correctness of the translated code.
- Understanding the core algorithm is more important than language syntax.